1 Dasar Input/Output (I/O)

Bab ini membahas mekanisme fundamental input/output di sistem Linux, termasuk konsep file descriptor, pengalihan I/O standar, penggunaan pipa untuk menghubungkan proses, dan berbagai teknik manipulasi aliran data. Pemahaman tentang I/O redirection sangat penting untuk otomasi dan scripting di Linux.

1.1 Filosofi I/O Unix: "Everything is a File"

1.1.1 Apa Itu Filosofi Unix I/O?

Salah satu prinsip fundamental dalam desain Unix (dan turunannya seperti Linux) adalah konsep **"everything is a file"**. Filosofi ini berarti bahwa hampir semua sumber daya sistem—file reguler, direktori, perangkat keras, socket jaringan, pipe, bahkan proses—diperlakukan sebagai file yang dapat dibaca atau ditulis.

Definisi 1.1 (Everything is a File)**.** Paradigma desain sistem operasi di mana berbagai jenis sumber daya sistem (hardware, software, komunikasi) diakses melalui antarmuka file yang seragam, menggunakan operasi standar seperti `open()`, `read()`, `write()`, dan `close()`.

1.1.2 Mengapa Filosofi Ini Penting?

Pendekatan "everything is a file" memberikan beberapa keuntungan arsitektural:

1. **Kesederhanaan Interface**: Programmer hanya perlu mempelajari satu set system call untuk mengakses berbagai jenis sumber daya
2. **Komposabilitas**: Program dapat digabungkan dengan mudah menggunakan pipe dan redirection karena semuanya menggunakan aliran byte yang sama
3. **Portabilitas**: Abstraksi ini menyembunyikan detail implementasi perangkat keras, membuat kode lebih portable

4. **Fleksibilitas**: Mudah mengganti sumber I/O (misalnya dari keyboard ke file) tanpa mengubah logika program

1.1.3 Kapan Filosofi Ini Diterapkan?

Filosofi ini tercermin dalam berbagai aspek Linux:

- **Perangkat Hardware**: `/dev/sda` (hard disk), `/dev/tty` (terminal), `/dev/null` (null device)
- **Informasi Proses**: `/proc/[PID]/` berisi informasi proses yang dapat dibaca seperti file
- **Informasi Sistem**: `/sys/` berisi informasi kernel dan perangkat
- **Inter-Process Communication**: Named pipes (FIFO) dan Unix domain sockets

Contoh: Membaca Informasi CPU

```
# Informasi CPU dapat dibaca seperti file biasa
cat /proc/cpuinfo

# Informasi memory
cat /proc/meminfo

# Semua menggunakan interface "file" yang sama
```

1.2 File Descriptor di Linux

1.2.1 Apa Itu File Descriptor?

Di Linux, setiap proses memiliki akses ke file dan stream melalui mekanisme yang disebut **file descriptor** (FD). File descriptor adalah integer non-negatif yang berfungsi sebagai *handle* atau *reference* untuk mengakses file, socket, pipe, atau perangkat I/O lainnya.

1.2.2 Mengapa File Descriptor Diperlukan?

File descriptor adalah implementasi konkret dari filosofi "everything is a file". Kernel menggunakan file descriptor untuk:

1. **Abstraksi**: Menyembunyikan detail implementasi perangkat keras dan sistem file dari program
2. **Keamanan**: Kernel mengontrol akses melalui file descriptor, memastikan proses hanya mengakses file yang diizinkan
3. **Efisiensi**: File descriptor adalah integer sederhana yang murah untuk di-copy dan di-pass antar fungsi
4. **Multiplexing**: Memungkinkan satu proses mengakses banyak file sekaligus dengan FD yang berbeda

Catatan

File descriptor adalah abstraksi yang memungkinkan program berinteraksi dengan berbagai jenis I/O menggunakan interface yang seragam, tanpa perlu tahu apakah I/O tersebut berasal dari file, keyboard, jaringan, atau perangkat lain.

1.2.3 Bagaimana File Descriptor Bekerja?

Secara internal, kernel memelihara tabel file descriptor untuk setiap proses:

1. Ketika proses membuka file dengan `open()`, kernel mengalokasikan FD berikutnya yang tersedia
2. Kernel membuat entri di **file descriptor table** yang menunjuk ke **open file table**
3. Open file table berisi informasi seperti file offset, mode akses, dan pointer ke inode
4. Program menggunakan FD (integer) untuk operasi selanjutnya, kernel menerjemahkannya ke file yang sesuai

Tips

File descriptor dialokasikan dari angka terkecil yang tersedia. Setelah 0, 1, 2 (stdin, stdout, stderr), FD berikutnya yang dibuka biasanya adalah 3, kemudian 4, dan seterusnya.

1.2.4 File Descriptor Standar

Setiap proses di Linux secara otomatis mendapat tiga file descriptor standar saat pertama kali dijalankan. Ketiga file descriptor ini selalu ada dan sudah dibuka oleh sistem operasi, sehingga setiap program dapat langsung menggunakannya tanpa perlu membuka file terlebih dahulu.

Tabel berikut menjelaskan ketiga file descriptor standar yang tersedia:

Tabel 1.1: File Descriptor Standar di Linux

FD	Nama	Simbol	Fungsi
0	Standard Input	`stdin`	Input dari keyboard/terminal
1	Standard Output	`stdout`	Output normal program
2	Standard Error	`stderr`	Pesan error dan diagnostik

Sebagai contoh, ketika Anda menjalankan perintah seperti `ls` di terminal, perintah tersebut membaca input dari `stdin` (FD 0, keyboard Anda), menampilkan hasil ke `stdout` (FD 1, layar terminal), dan jika ada error akan ditampilkan ke `stderr` (FD 2, juga ke layar terminal, tapi bisa dialihkan terpisah).

1.2.5 Melihat File Descriptor

Untuk memahami file descriptor dengan lebih baik, kita dapat melihat file descriptor yang sedang terbuka oleh suatu proses. Linux menyediakan sistem file virtual `/proc/` yang berisi informasi tentang proses yang sedang berjalan.

File descriptor yang terbuka oleh proses dapat dilihat di direktori `/proc/PID/fd/`, dimana PID adalah Process ID dari proses tersebut. Mari kita lihat contohnya:

```
# Melihat file descriptor dari proses bash saat ini
ls -l /proc/$$/fd

# Contoh output:
# lrwx------ 1 user user 0 Feb 16 10:00 0 -> /dev/pts/0
# lrwx------ 1 user user 0 Feb 16 10:00 1 -> /dev/pts/0
# lrwx------ 1 user user 0 Feb 16 10:00 2 -> /dev/pts/0
```

Kode 1.1: Melihat file descriptor proses

Penjelasan output di atas:

- Simbol `->` menunjukkan kemana file descriptor tersebut mengarah

- `/dev/pts/0` adalah pseudo-terminal (terminal virtual Anda saat ini)
- Ketiga file descriptor (0, 1, 2) mengarah ke tempat yang sama, yaitu terminal Anda
- Ini berarti input diambil dari keyboard terminal, dan output ditampilkan ke layar terminal yang sama

Tips

Variabel `$$` dalam bash adalah variabel spesial yang menyimpan PID (Process ID) dari shell saat ini. Jadi `$$` akan otomatis diganti dengan angka PID shell Anda.

1.3 Pengalihan I/O Standar

Pengalihan I/O (I/O redirection) adalah teknik untuk mengubah sumber input atau tujuan output dari perintah. Ini adalah salah satu fitur paling powerful dari sistem Unix-like, yang memungkinkan pemrosesan data yang fleksibel dan otomasi.

1.3.1 Apa Itu Pengalihan I/O?

Pengalihan I/O memungkinkan Anda untuk:

- Mengubah dari mana program membaca inputnya (selain dari keyboard)
- Mengubah ke mana program menulis outputnya (selain ke layar)
- Memisahkan output normal dari pesan error
- Menghubungkan beberapa program untuk memproses data

1.3.2 Mengapa Pengalihan I/O Penting?

Tanpa pengalihan I/O, setiap program harus mengimplementasikan kode file handling sendiri. Dengan redirection:

1. **Otomasi**: Script dapat memproses file tanpa interaksi user
2. **Logging**: Menyimpan output program untuk analisis kemudian
3. **Pemrosesan Data**: Menghubungkan program sederhana menjadi workflow kompleks
4. **Operasi Batch**: Memproses banyak file menggunakan perintah yang sama

Misalnya, daripada mengetik data secara manual ke perintah `sort`, Anda bisa mengalihkannya untuk membaca dari file yang berisi ribuan baris. Daripada menonton output scroll, Anda bisa menyimpannya ke file dan menganalisisnya nanti.

1.3.3 Standard Input (stdin)

Standard input (stdin) adalah sumber data untuk program. Secara default, stdin dibaca dari keyboard—artinya program menunggu Anda mengetik sesuatu. Namun, kita bisa mengalihkan stdin agar membaca dari file, sehingga program mendapat data dari file tersebut bukan dari keyboard.

Apa Itu Pengalihan stdin?

Pengalihan stdin mengubah sumber data input dari interaktif (keyboard) ke otomatis (file atau program lain). Ini fundamental untuk:

- Memproses dataset besar
- Mengotomasi tugas berulang
- Batch processing file
- Testing program dengan input yang sudah diketahui

Kapan Menggunakan Pengalihan stdin?

Gunakan pengalihan stdin ketika:

- Anda memiliki data input yang sudah ada dalam file
- Input terlalu besar untuk diketik manual
- Anda perlu menjalankan input yang sama melalui program berbeda
- Anda mengotomasi tugas yang biasanya memerlukan input user

Cara Mengalihkan stdin dari File

Operator `<` (tanda lebih kecil) digunakan untuk mengalihkan stdin dari file. Operator ini memberi tahu shell untuk mengambil input dari file, bukan dari keyboard.

Contoh penggunaan:

```
# Membaca input dari file.txt alih-alih keyboard
sort < file.txt
```

```
# Menghitung jumlah baris dari file
wc -l < /etc/passwd
```

Kode 1.2: Pengalihan stdin dari file

Penjelasan contoh di atas:

- Baris pertama: perintah `sort` akan membaca isi `file.txt` (bukan menunggu Anda mengetik), lalu mengurutkannya dan menampilkan hasilnya
- Baris kedua: perintah `wc -l` akan menghitung jumlah baris dari file `/etc/passwd`. Operator `-l` artinya "line count" (hitung baris)

Here Document («)

Here document (sering disingkat "heredoc") adalah fitur yang memungkinkan kita memberikan input multi-baris (banyak baris) langsung dalam script atau command line, tanpa perlu membuat file terpisah.

Masalah Apa yang Dipecahkan Here Document? Sebelum ada here document, jika Anda perlu memberikan input multi-baris ke program, Anda harus:

1. Membuat file temporary
2. Menulis konten ke file
3. Redirect file sebagai input
4. Menghapus file temporary

Here document menghilangkan kompleksitas ini dengan memungkinkan inline multi-line input.

Kapan Menggunakan Here Document? Gunakan here document ketika:

- Membuat file konfigurasi dalam script
- Membuat pesan email atau dokumen multi-baris
- Memberikan SQL query atau perintah multi-baris lainnya
- Menyematkan dokumentasi atau help text dalam script
- Testing program dengan input multi-baris tertentu

Mengapa Menggunakan Here Document Daripada echo? Meskipun Anda bisa menggunakan multiple perintah `echo`, here document lebih baik karena:

- Lebih mudah dibaca—konten terlihat seperti teks asli, bukan kode
- Mempertahankan formatting (line breaks, indentasi)
- Tidak perlu escape karakter spesial di setiap baris
- Satu blok teks lebih mudah dimaintain

Cara Kerja Here Document Kita menggunakan operator « diikuti dengan delimiter (penanda akhir, biasanya kata "EOF" yang berarti "End Of File"). Semua baris setelahnya akan dianggap sebagai input, sampai shell menemukan delimiter yang sama di baris terpisah.

Berikut contoh penggunaan:

```
# Mengirim email dengan heredoc
cat << EOF
To: user@example.com
Subject: Test Email

Ini adalah isi email.
Baris kedua.
EOF

# Membuat file konfigurasi
cat << 'EOF' > config.txt
server=localhost
port=8080
EOF
```

Kode 1.3: Menggunakan here document

Penjelasan contoh di atas:

- Contoh pertama: semua teks antara « `EOF` dan `EOF` akan menjadi input untuk perintah `cat`, yang kemudian ditampilkan ke layar
- Contoh kedua: menggunakan tanda kutip pada delimiter `'EOF'` untuk mencegah ekspansi variabel. Output disimpan ke file `config.txt` menggunakan operator `>`
- Delimiter bisa menggunakan kata apapun (EOF, END, STOP, dll), yang penting kata pembuka dan penutup harus sama

1.3.4 Standard Output (stdout)

Standard output (stdout) adalah tujuan output normal dari program. Ketika program menghasilkan output (seperti `ls`, `echo`, `cat`), secara default output tersebut ditampilkan ke terminal (layar Anda). Namun kita bisa mengalihkan output ini ke file menggunakan operator redirection.

Apa Itu Pengalihan stdout?

Pengalihan stdout menangkap output program dan menyimpannya ke file alih-alih menampilkannya di layar. Ini essential untuk:

- Membuat catatan permanen dari output command
- Membuat laporan dan log
- Menyimpan data untuk pemrosesan selanjutnya
- Mengotomasi tugas tanpa mengotori layar

Kapan Mengalihkan stdout?

Anda harus mengalihkan stdout ketika:

- Perlu menyimpan output command untuk dokumentasi
- Membuat log aktivitas sistem
- Menghasilkan data file untuk pemrosesan lebih lanjut
- Menjalankan command di background/cron jobs (tidak ada yang melihat layar)
- Output terlalu besar untuk dibaca di layar
- Ingin membandingkan output dari berbagai run

Operator > (Overwrite)

Operator > (tanda lebih besar) digunakan untuk mengalihkan stdout ke file. Jika file sudah ada, isinya akan **ditimpa** (dihapus dan diganti). Jika file belum ada, file baru akan dibuat.

Kapan Menggunakan Overwrite (>)? Gunakan operator overwrite ketika:

- Anda ingin output fresh setiap kali (data lama tidak diperlukan)
- Membuat laporan yang harus mengganti versi lama

- Membuat output file dalam kondisi bersih
- Data lama menjadi usang setelah setiap run

Contoh Berikut contoh penggunaan:

```
# Menulis output ke file (menimpa jika ada)
ls -la > daftar-file.txt

# Membuat file kosong
> empty-file.txt

# Menyimpan konfigurasi sistem
uname -a > system-info.txt
```

Kode 1.4: Pengalihan stdout dengan overwrite

Penjelasan contoh di atas:

- Baris pertama: output dari `ls -la` (daftar file lengkap) disimpan ke file `daftar-file.txt`
- Baris kedua: membuat file kosong bernama `empty-file.txt`. Operator `>` tanpa command akan membuat file kosong
- Baris ketiga: informasi sistem dari `uname -a` disimpan ke `system-info.txt`

Peringatan

Operator `>` akan menghapus isi file yang sudah ada! Jika file `daftar-file.txt` sudah berisi data, data lama akan hilang dan diganti dengan data baru. Gunakan dengan hati-hati.

Operator » (Append)

Operator » (dua tanda lebih besar) digunakan untuk mengalihkan stdout ke file dengan mode **append** (menambahkan). Berbeda dengan `>`, operator ini tidak menghapus isi file yang sudah ada, melainkan menambahkan output baru di akhir file.

Apa Itu Mode Append? Mode append mempertahankan konten file yang sudah ada dan menambahkan data baru di akhir. Ini berbeda fundamental dari mode overwrite karena:

- Data lama tetap utuh
- Data baru ditambahkan secara berurutan
- File bertambah besar seiring waktu
- Sempurna untuk mengakumulasi informasi

Kapan Menggunakan Append (»)? Gunakan operator append ketika:

- Membuat atau mengupdate file log (perlu menyimpan history)
- Mengakumulasi hasil dari beberapa run script
- Membangun laporan secara incremental
- Merekam event secara kronologis
- Menambah data ke file yang sudah ada tanpa kehilangan data lama

Mengapa Append Penting untuk Log? File log memerlukan mode append karena:

1. **Catatan Historis**: Anda perlu melihat apa yang terjadi dari waktu ke waktu, tidak hanya event terakhir
2. **Troubleshooting**: Masalah mungkin disebabkan oleh event berjam-jam atau berhari-hari lalu
3. **Audit Trail**: Compliance dan keamanan memerlukan history lengkap
4. **Analisis Pattern**: Anda bisa menganalisis trend dan pattern di berbagai run

Contoh Berikut contoh penggunaan:

```
# Menambahkan output ke file
date >> log.txt
echo "Proses selesai" >> log.txt

# Membuat log sederhana
echo "$(date): System started" >> /var/log/myapp.log
```

Kode 1.5: Pengalihan stdout dengan append

Penjelasan contoh di atas:

- Baris pertama: tanggal dan waktu saat ini ditambahkan ke akhir file `log.txt`
- Baris kedua: teks "Proses selesai" ditambahkan ke baris baru di `log.txt`, tanpa menghapus tanggal yang sudah ada

- Baris ketiga: membuat log entry dengan timestamp yang ditambahkan ke file log aplikasi

1.3.5 Standard Error (stderr)

Standard error (stderr) adalah channel terpisah yang khusus digunakan untuk pesan error dan diagnostik.

Apa Itu stderr dan Mengapa Ia Ada?

Desainer Unix menciptakan channel terpisah untuk error karena alasan yang sangat bagus:

1. **Separation of Concerns**: Output normal (data) dan error (diagnostik) melayani tujuan berbeda
2. **Integritas Pipeline**: Error tidak merusak data yang mengalir melalui pipe
3. **Penanganan Error Fleksibel**: Anda bisa menangani error berbeda dari data
4. **User Experience**: User bisa melihat error bahkan ketika output dialihkan

Masalah Tanpa stderr Bayangkan jika error dan output normal ke channel yang sama:

- Error akan tercampur ke dalam file data Anda
- Anda tidak bisa menyimpan data ke file sambil tetap melihat error di layar
- Pipeline akan rusak karena error diperlakukan sebagai data
- Script otomatis tidak bisa membedakan antara sukses dan gagal

Kapan Menggunakan Pengalihan stderr? Alihkan stderr secara terpisah ketika:

- Ingin menyimpan error ke file log terpisah untuk debugging
- Perlu membuang error yang diketahui/expected tanpa kehilangan output
- Ingin melihat error di layar sambil menyimpan output ke file
- Perlu penanganan berbeda untuk error vs. output normal
- Membuat alert atau notifikasi berdasarkan error

Mengapa Memisahkan Error dan Output? Manfaat penanganan error terpisah:

1. **Debugging Lebih Mudah**: Error ada di file sendiri, mudah ditemukan
2. **Data Bersih**: File output hanya berisi data, tanpa pesan error
3. **Automation-Friendly**: Script bisa mendeteksi dan menangani error secara programmatic
4. **Monitoring**: Anda bisa memonitor error log terpisah dari data

Cara Mengalihkan stderr

Untuk mengalihkan stderr, kita menggunakan operator `2>`. Angka 2 merujuk pada file descriptor untuk stderr (ingat: 0=stdin, 1=stdout, 2=stderr).

Contoh Berikut contoh penggunaan:

```
# Menyimpan error ke file
find / -name "*.conf" 2> errors.txt

# Membuang error (redirect ke /dev/null)
grep "pattern" file.txt 2> /dev/null

# Append error ke file
command 2>> error-log.txt
```

Kode 1.6: Pengalihan stderr

Penjelasan contoh di atas:

- Baris pertama: perintah `find` mencari di seluruh sistem (`//`). Akan ada banyak error "Permission denied" untuk folder yang tidak bisa diakses. Error-error ini disimpan ke `errors.txt`, sedangkan hasil pencarian yang berhasil tetap ditampilkan ke layar
- Baris kedua: error dari `grep` (misalnya "file not found") dibuang ke `/dev/null` (tempat sampah sistem). Berguna jika kita tahu error tidak penting
- Baris ketiga: error ditambahkan (append) ke file log, mirip dengan » untuk stdout

Mengalihkan stdout dan stderr

Dalam praktik nyata, seringkali kita perlu mengalihkan stdout dan stderr secara bersamaan. Situasi ini sangat umum terjadi ketika:

- Menjalankan script otomatis (cron jobs) yang perlu mencatat semua output
- Melakukan instalasi software yang menghasilkan banyak output dan mungkin error
- Debugging program yang kompleks dengan banyak pesan
- Membuat laporan lengkap dari eksekusi program

Mengapa kita perlu mengalihkan keduanya? Bayangkan Anda menjalankan script backup yang berjalan di malam hari. Jika ada error, Anda ingin tahu apa errornya (dari stderr). Tapi Anda juga ingin tahu file mana saja yang berhasil di-backup (dari stdout). Dengan mengalihkan keduanya ke file, Anda bisa melihat complete picture dari apa yang terjadi.

Ada beberapa cara untuk melakukannya, tergantung kebutuhan. Mari kita lihat empat metode yang paling umum digunakan:

```
# Metode 1: Redirect ke file yang berbeda
command > output.txt 2> error.txt

# Metode 2: Redirect keduanya ke file yang sama
command > all-output.txt 2>&1

# Metode 3: Syntax modern (Bash 4+)
command &> all-output.txt

# Metode 4: Append keduanya
command >> all-output.txt 2>&1
```

Kode 1.7: Mengalihkan stdout dan stderr

Sekarang mari kita bahas setiap metode secara detail:

Metode 1: File Terpisah untuk Output dan Error Metode ini memisahkan output normal dan error ke dua file berbeda:

```
# Instalasi package dengan log terpisah
sudo apt install nginx > install-output.txt 2> install-
   errors.txt

# Compile program
gcc program.c -o program > compile-output.txt 2> compile-
   errors.txt
```

Kode 1.8: Contoh metode 1 - file terpisah

Kapan menggunakan Metode 1?

- Ketika debugging: Anda bisa langsung membuka file error tanpa harus mencari di antara output normal
- Ketika error perlu diproses terpisah (misalnya dikirim email ke admin)
- Ketika output normal sangat banyak dan error jarang, sehingga error mudah terlewat jika dicampur

Cara kerjanya:

1. `>` mengalihkan stdout (FD 1) ke `output.txt`
2. `2>` mengalihkan stderr (FD 2) ke `error.txt`
3. Kedua operator bekerja independen
4. Jika tidak ada error, file `error.txt` akan kosong

Metode 2: File yang Sama (Metode Klasik) Metode ini menggabungkan stdout dan stderr ke satu file yang sama. Syntax-nya: `command > file 2>&1`

```
# Backup dengan log lengkap
rsync -av /home /backup > backup.log 2>&1

# Running test dengan semua output
./run-tests.sh > test-results.log 2>&1
```

Kode 1.9: Contoh metode 2 - file yang sama

Kapan menggunakan Metode 2?

- Ketika ingin satu file log lengkap yang mencakup semua
- Ketika urutan kronologis output dan error penting (mereka muncul sesuai urutan waktu)
- Untuk kompatibilitas dengan versi Bash lama (bekerja di semua versi)

Cara kerjanya (PENTING - urutan matters!):

1. `> all-output.txt`: Alihkan stdout ke file
2. `2>&1`: Alihkan stderr ke "tempat yang sama dengan stdout" (yang sudah mengarah ke file)

3. Karakter `&` penting! Tanpa `&`, shell akan mengira "1" adalah nama file
4. **Urutan harus benar**: `> file` dulu, baru `2>&1`

Metode 3: Syntax Modern dengan &> Bash versi 4 ke atas menyediakan shortcut: `&>`

```
# Lebih ringkas dan mudah dibaca
./deployment-script.sh &> deploy.log

# Sama hasilnya dengan metode 2, tapi lebih sederhana
docker build -t myapp . &> build.log
```

Kode 1.10: Contoh metode 3 - syntax modern

Kapan menggunakan Metode 3?

- Ketika Anda yakin menggunakan Bash versi 4+ (check dengan: `bash -version`)
- Ketika ingin syntax yang lebih mudah dibaca dan ditulis
- Untuk script baru yang tidak perlu kompatibilitas dengan shell lama

Cara kerjanya:

- `&>` adalah operator khusus yang langsung mengalihkan keduanya
- Equivalent dengan `> file 2>&1`, tapi lebih ringkas
- Tidak bisa dipakai untuk append (tidak ada `&>>`)

Metode 4: Append Keduanya Ketika Anda ingin menambahkan ke file yang sudah ada (tidak menghapus isi lama):

```
# Running script berkali-kali, accumulate semua log
./daily-report.sh >> reports.log 2>&1

# Cron job yang running setiap hari
0 2 * * * /usr/local/bin/backup.sh >> /var/log/backup.log
   2>&1
```

Kode 1.11: Contoh metode 4 - append keduanya

Peringatan

Kesalahan Umum: Menulis `2>&1 > file` (urutan terbalik)
Ini TIDAK akan bekerja dengan benar! Stderr akan tetap ke terminal, bukan ke file. Mengapa? Karena saat `2>&1` dieksekusi, stdout masih mengarah ke terminal. Baru setelah itu stdout dialihkan ke file, tapi stderr sudah "terlanjur" dikunci ke terminal.
Urutan yang benar: `> file 2>&1`

Kapan menggunakan Metode 4?

- Script yang dijalankan berulang kali (cron jobs)
- Log file yang perlu menyimpan history
- Debugging bertahap (menjalankan program berkali-kali sambil accumulate output)

Cara kerjanya:

- Sama seperti Metode 2, tapi menggunakan » untuk append
- » `file`: Append stdout ke file (tidak menghapus isi lama)
- `2>&1`: Stderr juga ikut append ke tempat yang sama
- File akan terus bertambah besar—perlu log rotation untuk production

Perbandingan Praktis: Kapan Pakai Yang Mana? Berikut panduan cepat memilih metode:

- **Debugging program**: Gunakan Metode 1 (file terpisah) agar error mudah ditemukan
- **Script sekali jalan**: Gunakan Metode 3 (`&>`) jika Bash 4+, atau Metode 2 jika perlu kompatibilitas
- **Cron jobs / recurring tasks**: Gunakan Metode 4 (append) agar semua history tersimpan
- **Production systems**: Gunakan Metode 2 atau 4, dengan log rotation untuk manage ukuran file

Contoh: Memisahkan Output dan Error

Untuk memahami penggunaan redirection dalam situasi nyata, mari lihat contoh script backup yang mencatat output normal dan error ke file terpisah:

```
#!/bin/bash
# Jalankan backup dengan logging

backup_dir="/backup"
log_file="/var/log/backup.log"
error_file="/var/log/backup-error.log"

echo "$(date): Memulai backup..." >> "$log_file"

# Jalankan rsync dengan output terpisah
rsync -av /home/ "$backup_dir" \
    >> "$log_file" 2>> "$error_file"

if [ $? -eq 0 ]; then
    echo "$(date): Backup berhasil" >> "$log_file"
else
    echo "$(date): Backup gagal" >> "$log_file"
fi
```

Penjelasan script di atas:

- Script menggunakan » untuk append karena kita ingin menyimpan history backup
- Output normal dari `rsync` (daftar file yang di-backup) masuk ke `backup.log`
- Error (misalnya file yang tidak bisa dibaca) masuk ke `backup-error.log`
- Variabel `$?` berisi exit code dari command terakhir: 0 berarti sukses, bukan 0 berarti error
- Dengan pemisahan ini, kita bisa dengan mudah melihat apakah ada error tanpa harus membaca semua output

Setelah memahami berbagai operator redirection, berikut ringkasan lengkap semua operator yang tersedia:

Tabel 1.2: Operator Pengalihan I/O

Operator	Deskripsi
<	Redirect stdin dari file
>	Redirect stdout ke file (overwrite)
»	Redirect stdout ke file (append)
2>	Redirect stderr ke file (overwrite)
2»	Redirect stderr ke file (append)
2>&1	Redirect stderr ke stdout
&>	Redirect stdout dan stderr ke file (Bash 4+)
«	Here document (input multi-baris)
«<	Here string (input string tunggal)

1.4 Pipa dan Operator |

1.4.1 Apa Itu Pipe?

Pipe (pipa) adalah mekanisme Inter-Process Communication (IPC) yang menghubungkan stdout dari satu perintah ke stdin perintah lain. Operator pipe adalah `|`.

Definisi 1.2 (Pipe)**.** Saluran komunikasi satu arah (unidirectional) yang menghubungkan output suatu proses sebagai input proses lain, menciptakan aliran data yang seamless tanpa perlu file intermediet.

1.4.2 Mengapa Pipe Penting dalam Unix?

Pipe adalah implementasi kunci dari **Unix Philosophy**: "Write programs that do one thing and do it well. Write programs to work together."

1. **Modularitas**: Program kecil yang fokus pada satu tugas dapat dikombinasikan untuk tugas kompleks
2. **Efisiensi**: Data mengalir langsung antar proses tanpa menulis ke disk, menghemat I/O
3. **Paralelisme**: Proses dalam pipeline berjalan secara concurrent, meningkatkan throughput
4. **Reusabilitas**: Program yang sama dapat digunakan dalam berbagai kombinasi pipeline

Catatan

Pipe menciptakan buffer di kernel (biasanya 64KB di Linux modern). Jika buffer penuh, proses penulis akan blocked sampai proses pembaca mengambil data. Ini adalah bentuk natural flow control.

1.4.3 Kapan Menggunakan Pipe?

Gunakan pipe ketika:

- Anda perlu memproses data melalui serangkaian transformasi
- Output satu program cocok sebagai input program lain
- Anda ingin menghindari file temporary
- Anda perlu memproses stream data yang besar (tidak perlu load semua ke memory)
- Anda ingin memanfaatkan paralelisme (pipeline stages berjalan bersamaan)

Tips

Pipe sangat efisien untuk data besar karena menggunakan streaming: data diproses chunk-by-chunk, bukan sekaligus. Ini memungkinkan pemrosesan file gigabyte dengan memory minimal.

1.4.4 Konsep Pipa

Pipa memungkinkan aliran data langsung antar proses tanpa file perantara. Bayangkan seperti pipa air: air mengalir dari satu ujung ke ujung lain tanpa tumpah. Dalam konteks Linux, "air" tersebut adalah data (text output).

Mari lihat contoh dasar penggunaan pipe:

```
# Menampilkan 10 file terbesar
ls -lh | sort -k5 -rh | head -10

# Menghitung jumlah user di sistem
cat /etc/passwd | wc -l

# Mencari proses tertentu
ps aux | grep apache
```

Kode 1.12: Penggunaan dasar pipe

Penjelasan contoh di atas:

1. **Contoh pertama** - menampilkan 10 file terbesar:
   - `ls -lh`: tampilkan daftar file dengan ukuran human-readable (KB, MB, dll)
   - `|`: kirim output ke command berikutnya
   - `sort -k5 -rh`: urutkan berdasarkan kolom ke-5 (ukuran file), `-r`=reverse (besar ke kecil), `-h`=human-readable numbers
   - `|`: kirim hasil sorting ke command berikutnya
   - `head -10`: ambil 10 baris pertama saja
2. **Contoh kedua** - menghitung user:
   - `cat /etc/passwd`: tampilkan isi file passwd (berisi daftar user, satu user per baris)
   - `|`: kirim ke command berikutnya
   - `wc -l`: hitung jumlah baris (`-l` = line count)
3. **Contoh ketiga** - cari proses:
   - `ps aux`: tampilkan semua proses yang sedang berjalan
   - `|`: kirim ke command berikutnya
   - `grep apache`: filter hanya baris yang mengandung kata "apache"

1.4.5 Rantai Pipa

Kekuatan sebenarnya dari pipe terlihat ketika kita menghubungkan beberapa perintah dalam satu rantai panjang (pipeline). Setiap program dalam pipeline melakukan satu tugas kecil, dan hasilnya digabungkan untuk tugas yang kompleks.

Mari lihat contoh pipeline yang lebih kompleks:

```
# Analisis log: cari error, hitung, dan urutkan
cat /var/log/syslog | grep "error" | \
    awk '{print $5}' | sort | uniq -c | sort -rn

# Monitoring penggunaan disk real-time
df -h | grep "^/dev" | \
    awk '{print $5, $6}' | sort -rn

# Mencari file terbesar di home directory
```

```
find ~ -type f -exec ls -lh {} \; | \
    awk '{print $5, $9}' | sort -rh | head -20
```

Kode 1.13: Rantai pipa (pipeline)

Penjelasan pipeline di atas:

1. **Analisis log error**:
   - `cat /var/log/syslog`: baca file log sistem
   - `grep "error"`: filter hanya baris yang ada kata "error"
   - `awk '{print $5}'`: ambil kolom ke-5 (biasanya nama service/program)
   - `sort`: urutkan alphabetically
   - `uniq -c`: hitung berapa kali setiap baris muncul
   - `sort -rn`: urutkan berdasarkan angka (`-n`), terbesar dulu (`-r`)
   - Hasil: daftar service mana yang paling banyak menghasilkan error
2. **Monitoring disk**: Menampilkan penggunaan disk yang terurut dari yang paling penuh
3. **File terbesar**: Menemukan 20 file terbesar di home directory Anda

Catatan

Backslash (\) di akhir baris memungkinkan perintah dilanjutkan ke baris berikutnya untuk keterbacaan yang lebih baik. Ini sangat berguna untuk pipeline panjang agar mudah dibaca.

1.4.6 Pipa vs Redirection

Perbedaan utama antara pipa dan redirection:

- **Pipa** (`|`): Menghubungkan stdout ke stdin proses lain
- **Redirection** (`>`): Menghubungkan stdout ke file

```
# Pipa untuk filter, redirect untuk menyimpan
ps aux | grep apache | sort -k3 -rn > apache-processes.txt

# Membuat laporan dengan timestamp
date > report.txt
df -h | grep "^/dev" >> report.txt
echo "---" >> report.txt
```

```
free -h >> report.txt
```

Kode 1.14: Kombinasi pipa dan redirection

1.4.7 Named Pipes (FIFO)

Named pipe (juga disebut FIFO = First In First Out) adalah jenis pipe spesial yang memiliki nama dan bisa diakses seperti file biasa. Berbeda dengan pipe biasa yang hilang setelah command selesai, named pipe tetap ada di filesystem sampai dihapus.

Named pipe berguna untuk komunikasi antar proses yang berjalan terpisah atau di terminal yang berbeda.

Berikut cara membuat dan menggunakan named pipe:

```
# Membuat named pipe
mkfifo mypipe

# Terminal 1: Menulis ke pipe
echo "Hello from terminal 1" > mypipe

# Terminal 2: Membaca dari pipe
cat < mypipe

# Menghapus named pipe
rm mypipe
```

Kode 1.15: Menggunakan named pipe

Contoh: Pipeline untuk Analisis Data

Contoh penggunaan pipeline untuk menganalisis log akses web:

```
# Analisis log Apache: IP dengan akses terbanyak
cat /var/log/apache2/access.log | \
    awk '{print $1}' | \
    sort | \
    uniq -c | \
    sort -rn | \
    head -10

# Mencari URL yang paling sering diakses
cat /var/log/apache2/access.log | \
    awk '{print $7}' | \
    sort | \
    uniq -c | \
    sort -rn | \
    head -20 > top-urls.txt

# Menghitung bandwidth per IP
cat /var/log/apache2/access.log | \
    awk '{bytes[$1]+=$10} END {for (ip in bytes)
   print ip, bytes[ip]}' | \
    sort -k2 -rn
```

1.5 Pengalihan Output ke File

Pengalihan output memungkinkan menyimpan hasil eksekusi program ke file untuk analisis atau dokumentasi.

1.5.1 Operator > (Overwrite)

Operator > membuat file baru atau menimpa file yang sudah ada:

```
# Menyimpan daftar proses
ps aux > processes.txt

# Membuat file konfigurasi
```

```
echo "DEBUG=true" > .env

# Backup daftar paket terinstal
dpkg -l > installed-packages.txt
```

Kode 1.16: Overwrite dengan operator >

1.5.2 Operator » (Append)

Operator » menambahkan output di akhir file:

```
# Membuat log harian
echo "$(date): System boot" >> /var/log/daily.log

# Menambahkan catatan
echo "TODO: Update documentation" >> notes.txt

# Mengumpulkan informasi sistem
echo "=== CPU Info ===" >> system-report.txt
cat /proc/cpuinfo >> system-report.txt
echo "=== Memory Info ===" >> system-report.txt
free -h >> system-report.txt
```

Kode 1.17: Append dengan operator »

1.5.3 Noclobber Option

Bash memiliki opsi `noclobber` untuk mencegah penimpaan file tidak sengaja:

```
# Aktifkan noclobber
set -o noclobber

# Ini akan gagal jika file ada
echo "data" > existing-file.txt  # Error jika file ada

# Paksa overwrite dengan >|
echo "data" >| existing-file.txt  # Berhasil meski
   noclobber aktif

# Nonaktifkan noclobber
```

```
set +o noclobber
```

Kode 1.18: Menggunakan noclobber

1.5.4 Null Device (/dev/null)

Apa Itu /dev/null?

`/dev/null` adalah file perangkat khusus di sistem Unix-like, sering disebut "bit bucket" atau "black hole" (lubang hitam). File ini memiliki properti unik:

- **Write**: Apa pun yang ditulis ke dalamnya langsung hilang
- **Read**: Membaca dari file ini tidak menghasilkan apa-apa (langsung EOF)
- **Selalu Tersedia**: File ini selalu ada di sistem
- **Tidak Pernah Penuh**: Anda bisa mengalihkan data tak terbatas ke dalamnya

Mengapa /dev/null Dibuat?

Desainer Unix menciptakan `/dev/null` untuk mengatasi masalah umum: kadang Anda perlu menjalankan perintah tetapi tidak peduli dengan outputnya. Tanpa `/dev/null`, Anda harus:

1. Membuat file sementara
2. Mengalihkan output ke file tersebut
3. Ingat untuk menghapus file tersebut nanti

Dengan `/dev/null`, Anda cukup mengalihkan output ke sana dan lupakan.

Kapan Menggunakan /dev/null?

Gunakan `/dev/null` ketika:

1. **Menekan Error yang Diharapkan**: Ketika menjalankan `find /` Anda akan mendapat banyak error "Permission denied" untuk direktori sistem yang tidak bisa diakses. Error ini diharapkan dan tidak berguna:

   ```
   find / -name "*.conf" 2> /dev/null
   ```

2. **Eksekusi Perintah Tanpa Suara**: Ketika menjalankan perintah di script yang outputnya tidak diperlukan:

```
# Cek apakah perintah berhasil, tidak peduli outputnya
if wget -q -O /dev/null http://example.com; then
    echo "Situs aktif"
fi
```

3. **Testing Perintah**: Ketika menguji syntax perintah tanpa mengacaukan layar
4. **Mengosongkan File**: Cara cepat untuk mengosongkan file (truncate ke 0 byte) tanpa menghapusnya

Mengapa Tidak Sekalian Hapus Error Messages?

Anda mungkin bertanya: mengapa redirect ke `/dev/null` daripada memperbaiki kode agar tidak menghasilkan error? Jawabannya:

- Beberapa error adalah **diharapkan dan tidak berbahaya** (seperti permission denied saat searching system-wide)
- Anda tidak mengontrol program yang menghasilkan error tersebut
- Error tersebut sebenarnya bukan error—hanya perilaku normal program
- Anda ingin program berjalan sampai selesai tanpa berhenti pada masalah kecil

Contoh

```
# Membuang output yang tidak diperlukan
find / -name "*.log" 2> /dev/null

# Menjalankan perintah tanpa output
command > /dev/null 2>&1

# Mengosongkan file (truncate ke 0 byte)
> /dev/null > large-file.log
# atau
cat /dev/null > large-file.log

# Test apakah file ada tanpa output
if [ -f /etc/passwd ] > /dev/null 2>&1; then
    echo "File ada"
```

```
fi
```

Kode 1.19: Menggunakan /dev/null

Tips

`/dev/null` sangat berguna ketika Anda ingin mengecek apakah perintah **berhasil** (exit code) tetapi tidak peduli apa outputnya. Exit code memberi tahu sukses/gagal; outputnya tidak relevan.

Peringatan

Hati-hati jangan sampai tidak sengaja mengalihkan output penting ke `/dev/null`! Sekali data masuk ke sana, hilang selamanya. Selalu periksa ulang operator redirection Anda.

1.6 Penggunaan tee untuk Output Ganda

1.6.1 Apa Itu tee?

Perintah `tee` membaca dari stdin dan menulis ke stdout **sekaligus** ke satu atau lebih file. Nama "tee" diambil dari bentuk huruf T yang menggambarkan aliran data yang terbagi dua (split).

1.6.2 Mengapa Menggunakan tee?

Tanpa `tee`, Anda harus memilih: lihat output di terminal ATAU simpan ke file. Dengan `tee`, Anda bisa melakukan keduanya:

1. **Real-time Monitoring**: Lihat progress program sambil merekam log
2. **Debugging**: Simpan output untuk analisis nanti tanpa kehilangan visibility
3. **Audit Trail**: Dokumentasikan apa yang terjadi saat maintenance/deployment
4. **Multiple Outputs**: Kirim data ke beberapa destinasi sekaligus

1.6.3 Kapan Menggunakan tee?

Gunakan `tee` dalam situasi berikut:

- **Instalasi/Update Sistem**: Monitor progress sambil simpan log
- **Deployment Scripts**: Lihat output sambil catat untuk audit
- **Long-running Commands**: Simpan output untuk review kemudian
- **Write ke Protected Files**: Bypass permission issue dengan `sudo tee`
- **Pipeline Debugging**: Lihat data intermediate di tengah pipeline

Catatan

`tee` tidak mengubah data yang melewatinya, hanya menduplikasi ke file. Ini membuatnya ideal untuk debugging pipeline tanpa mengubah behavior.

1.6.4 Syntax Dasar tee

```
# Tampilkan di layar DAN simpan ke file
command | tee output.txt

# Append ke file dengan -a
command | tee -a output.txt

# Menulis ke beberapa file sekaligus
command | tee file1.txt file2.txt file3.txt
```

Kode 1.20: Syntax dasar tee

1.6.5 Contoh Penggunaan tee

```
# Lihat output instalasi dan simpan log
sudo apt update | tee apt-update.log

# Monitoring dan logging
ping google.com | tee ping-log.txt

# Membuat backup sembari melihat progress
tar -czf backup.tar.gz /home/user | tee backup-log.txt

# Pipeline dengan tee
ps aux | tee processes.txt | grep apache
```

Kode 1.21: Contoh praktis tee

Penjelasan Contoh Mari kita pahami setiap contoh di atas:

1. **apt update dengan tee**: Ketika Anda menjalankan `sudo apt update`, Anda ingin melihat paket mana yang diupdate secara real-time, tetapi juga ingin menyimpan log untuk dokumentasi atau troubleshooting. Tanpa tee, Anda harus memilih salah satu.
2. **ping dengan tee**: Saat melakukan network monitoring dengan `ping`, Anda ingin melihat response time secara langsung untuk memastikan koneksi stabil, tetapi juga ingin merekam data untuk analisis nanti (misalnya untuk mendeteksi pola packet loss).
3. **backup dengan tee**: Saat membuat backup dengan `tar`, prosesnya bisa memakan waktu lama. Anda ingin melihat progress (file mana yang sedang di-compress), tetapi juga ingin log lengkap untuk verifikasi backup berhasil.
4. **Pipeline debugging**: Anda bisa "menyadap" data di tengah pipeline tanpa mengganggu aliran data. Data tetap mengalir ke `grep`, tetapi juga tersimpan di `processes.txt` untuk inspeksi.

1.6.6 Bagaimana tee Bekerja Secara Internal?

Untuk memahami `tee` lebih dalam, mari kita lihat bagaimana ia bekerja:

1. `tee` membaca data dari stdin (file descriptor 0)
2. Untuk setiap chunk data yang dibaca:
   - Tulis ke stdout (fd 1) - sehingga terminal atau pipe berikutnya menerima data
   - Tulis ke file(s) yang ditentukan - sehingga data tersimpan
3. Ulangi sampai input selesai (EOF)

Perbedaan dengan redirection:

- **Redirection** (>): Data hanya pergi ke file, tidak ke terminal
- **tee**: Data pergi ke KEDUANYA - terminal DAN file

1.6.7 tee dengan Multiple Files

Salah satu kekuatan `tee` adalah kemampuannya menulis ke beberapa file sekaligus:

```
# Simpan ke 3 file sekaligus
```

```
echo "Data penting" | tee file1.txt file2.txt file3.txt

# Use case: Log ke berbagai level
./deploy.sh 2>&1 | tee logs/deploy-full.log \
                       logs/deploy-summary.log \
                       /var/log/deploy.log

# Distribusi data ke multiple consumers
cat sensor-data.csv | tee raw-backup.csv |
    tee >(process-script1.sh > output1.txt) |
    process-script2.sh > output2.txt
```

Kode 1.22: tee dengan multiple files

Mengapa ini berguna?

- **Redundansi**: Backup ke multiple lokasi untuk disaster recovery
- **Multiple Formats**: Save raw data sekaligus processed data
- **Different Destinations**: Local file + network share + cloud storage
- **Access Control**: Copy ke direktori dengan permission berbeda

1.6.8 Perbedaan tee dan tee -a

Memahami kapan menggunakan mode overwrite vs append sangat penting:

```
# Mode overwrite (default): File baru setiap kali
date | tee timestamp.log     # File berisi HANYA timestamp
    terakhir

# Mode append (-a): Akumulasi data over time
date | tee -a timestamp.log  # File berisi SEMUA timestamp
```

Kode 1.23: Overwrite vs Append

Kapan menggunakan masing-masing?

Overwrite (`tee`) :

- Status checks yang tidak perlu history
- Output yang selalu fresh/current

- Temporary results yang di-replace tiap run

Append (`tee -a`) :

- Continuous monitoring (logs, metrics)
- Historical data accumulation
- Multiple script runs yang perlu combined
- Audit trails

1.6.9 Error Handling dengan tee

Penting untuk memahami bagaimana `tee` menangani error:

```
# Jika file tidak bisa ditulis, tee tetap output ke stdout
command | tee /read-only/file.txt  # Error, tapi stdout
   tetap jalan

# Capture exit code dari command, bukan tee
command | tee output.txt
echo $?  # Exit code dari TEE, bukan command!

# CORRECT: Gunakan PIPESTATUS
command | tee output.txt
echo ${PIPESTATUS[0]}  # Exit code dari command
echo ${PIPESTATUS[1]}  # Exit code dari tee
```

Kode 1.24: Error handling dengan tee

Peringatan

Variable `$?` di bash hanya menangkap exit code dari **command terakhir** di pipeline. Untuk mendapatkan exit code dari command di awal pipeline, gunakan array `PIPESTATUS`.

1.6.10 Use Cases Umum tee

1. Development dan Testing

```
# Run tests dan simpan output untuk CI/CD
npm test | tee test-results.txt
```

```
# Jika test gagal, bisa review log tanpa re-run
cat test-results.txt | grep "FAILED"
```

Kode 1.25: Testing dengan tee

2. System Administration

```
# Monitor disk usage setiap jam
watch -n 3600 'df -h | tee -a disk-usage-$(date +%F).log'

# Deployment dengan audit trail
./deploy-production.sh 2>&1 | tee -a /var/log/deployments.
   log
```

Kode 1.26: System admin dengan tee

3. Data Processing

```
# Process data, simpan raw dan processed
cat raw-data.csv |
    tee raw-backup.csv |
    process-script.py |
    tee processed-backup.csv |
    load-to-database.sh
```

Kode 1.27: Data processing dengan tee

Mengapa pattern ini powerful? Di contoh data processing di atas:

1. Data asli tersimpan di `raw-backup.csv` sebelum processing
2. Data diproses oleh `process-script.py`
3. Hasil processing tersimpan di `processed-backup.csv`
4. Data akhir dimuat ke database
5. Jika ada masalah di step mana pun, Anda punya backup untuk replay

1.6.11 Best Practices tee

1. **Naming Convention**: Gunakan timestamp di nama file log

```
command | tee "log-$(date +%Y%m%d-%H%M%S).txt"
```

2. **Disk Space**: Pastikan cukup ruang, terutama untuk long-running commands

```
df -h /var/log  # Cek space sebelum logging
```

3. **Suppress Terminal Output**: Redirect tee output ke /dev/null jika hanya butuh file

```
command | tee output.txt > /dev/null  # Hanya ke file,
    tidak ke terminal
```

4. **Buffering**: Gunakan `unbuffer` atau `stdbuf` untuk real-time output

```
# Untuk program yang buffer outputnya
unbuffer command | tee output.txt
# atau
stdbuf -oL command | tee output.txt
```

1.6.12 tee dengan sudo

Ketika perlu menulis ke file yang memerlukan privilege root:

```
# SALAH: Redirect tidak bekerja dengan sudo
sudo echo "option" > /etc/config.conf  # Gagal: permission
    denied

# BENAR: Gunakan tee
echo "option" | sudo tee /etc/config.conf

# Append dengan privilege root
echo "new line" | sudo tee -a /etc/hosts
```

```
# Tanpa output ke terminal
echo "secret" | sudo tee /etc/secret.conf > /dev/null
```

Kode 1.28: tee dengan sudo

Catatan

Operator redirect (>) diproses oleh shell sebelum `sudo`, sehingga tidak mendapat privilege root. Gunakan `tee` untuk menulis ke file yang memerlukan root.

1.6.13 tee dalam Script

Contoh: Logging Script dengan tee

Script yang mencatat output ke file dan terminal:

```
#!/bin/bash
# Deploy script dengan logging

LOGFILE="/var/log/deploy-$(date +%Y%m%d-%H%M%S).log"

# Semua output dicatat dengan tee
{
    echo "=== Deploy dimulai: $(date) ==="

    echo "Pulling latest code..."
    git pull origin main

    echo "Installing dependencies..."
    npm install

    echo "Building application..."
    npm run build

    echo "Restarting service..."
    sudo systemctl restart myapp

    echo "=== Deploy selesai: $(date) ==="
} 2>&1 | tee "$LOGFILE"

# Cek hasil
if [ ${PIPESTATUS[0]} -eq 0 ]; then
    echo "Deploy berhasil. Log: $LOGFILE"
else
    echo "Deploy gagal. Cek log: $LOGFILE"
fi
```

1.7 Teknik Lanjutan I/O

1.7.1 Process Substitution

Apa Itu Process Substitution?

Process substitution adalah fitur bash yang memungkinkan output suatu perintah muncul sebagai file. Bash menciptakan *named pipe* temporary dan menjalankan perintah, sehingga program lain bisa membacanya seperti file biasa.

Syntax: `<(command)` untuk input, `>(command)` untuk output.

Mengapa Menggunakan Process Substitution?

Beberapa program hanya menerima file sebagai argumen, bukan stdin. Process substitution mengatasi keterbatasan ini:

1. **Kompatibilitas**: Program yang perlu filename (seperti `diff`) bisa menerima output command
2. **Efisiensi**: Tidak perlu temporary file
3. **Multiple Inputs**: Bisa feed beberapa command output ke satu program

Kapan Menggunakan Process Substitution?

- Comparing outputs: `diff <(cmd1) <(cmd2)`
- Tools yang butuh filename: `vimdiff`, `comm`, `join`
- Menghindari temporary files untuk one-time use

Contoh Penggunaan

Process substitution memungkinkan menggunakan output perintah sebagai file:

```
# Membandingkan output dua perintah
diff <(ls dir1) <(ls dir2)

# Menggunakan output sebagai input
sort <(cat file1.txt file2.txt)

# Menggabungkan hasil beberapa perintah
```

```
paste <(cut -d: -f1 /etc/passwd) <(cut -d: -f3 /etc/passwd
    )
```

Kode 1.29: Process substitution

1.7.2 Command Substitution

Apa Itu Command Substitution?

Command substitution adalah teknik untuk menangkap output suatu perintah dan menggunakannya sebagai string dalam perintah lain atau menyimpannya dalam variabel.

Syntax modern: `$(command)` (lebih direkomendasikan)

Syntax lama: `‘command‘` (backticks, deprecated)

Mengapa Menggunakan Command Substitution?

1. **Dynamic Values**: Generate nilai berdasarkan kondisi sistem
2. **Automation**: Script dapat beradaptasi dengan environment
3. **Composability**: Gabungkan output beberapa perintah

Kapan Menggunakan Command Substitution?

- Membuat filename dynamic (dengan timestamp, hostname, dll)
- Menyimpan output untuk conditional logic
- Menggunakan output command sebagai argumen command lain
- Generating configuration based on system state

Catatan

Gunakan `$()` alih-alih backticks karena lebih readable dan mendukung nesting dengan lebih baik.

Contoh Penggunaan

Menangkap output perintah sebagai string:

```
# Syntax modern dengan $()
```

```
current_date=$(date +%Y-%m-%d)
echo "Tanggal: $current_date"

# Menggunakan dalam script
backup_file="backup-$(hostname)-$(date +%Y%m%d).tar.gz"

# Nested command substitution
echo "Ada $(cat /etc/passwd | wc -l) user di sistem"
```

Kode 1.30: Command substitution

1.7.3 File Descriptor Lanjutan

Membuat dan menutup file descriptor custom:

```
# Membuka file descriptor 3 untuk membaca
exec 3< input.txt

# Membaca dari FD 3
while read -u 3 line; do
    echo "Baris: $line"
done

# Menutup FD 3
exec 3<&-

# Membuka FD 4 untuk menulis
exec 4> output.txt
echo "Data" >&4
exec 4>&-
```

Kode 1.31: Custom file descriptor

1.8 Kapan Menggunakan Teknik I/O Tertentu?

Memilih teknik I/O yang tepat bergantung pada kebutuhan spesifik. Berikut panduan pemilihan:

1.8.1 Redirection vs Pipe

Tabel 1.3: Perbandingan Redirection dan Pipe

Aspek	Redirection (>)	Pipe (|)
Tujuan	File di disk	Proses lain (stdin)
Persistensi	Data tersimpan permanent	Data transient (hilang setelah diproses)
Gunakan ketika	Perlu simpan hasil untuk nanti	Perlu proses data lebih lanjut
Efisiensi	Butuh I/O disk	Fully in-memory (lebih cepat)
Contoh use case	Logging, backup output	Data transformation, filtering

1.8.2 Append (») vs Overwrite (>)

- **Gunakan > (overwrite)** ketika:
  - Membuat file output baru
  - Data lama tidak diperlukan
  - Ingin hasil yang clean dari awal
- **Gunakan » (append)** ketika:
  - Menambahkan ke log file
  - Mengakumulasi hasil dari multiple runs
  - Data lama harus dipertahankan

Peringatan

Salah menggunakan > alih-alih » pada log file akan menghapus semua history. Selalu double-check operator yang digunakan!

1.8.3 tee vs Redirection

- **Gunakan `tee`** ketika:
  - Perlu lihat output real-time DAN simpan ke file
  - Debugging pipeline (lihat intermediate data)
  - Multiple output destinations diperlukan

  - Perlu write ke file dengan `sudo` privilege

- **Gunakan redirection** ketika:
  - Hanya perlu simpan, tidak perlu lihat
  - Background process (tidak ada terminal)
  - Maksimalkan performa (tee sedikit lebih lambat)

1.8.4 Pipe vs Temporary File

Contoh: Perbandingan Pipe vs Temporary File

```
# Menggunakan temporary file (kurang efisien)
ps aux > /tmp/processes.txt
grep apache /tmp/processes.txt > /tmp/apache.txt
sort /tmp/apache.txt
rm /tmp/processes.txt /tmp/apache.txt

# Menggunakan pipe (lebih efisien)
ps aux | grep apache | sort
```

Keuntungan Pipe:

- Tidak ada I/O disk (lebih cepat)
- Tidak perlu cleanup temporary files
- Proses berjalan parallel (concurrent)
- Lebih ringkas dan readable

Keuntungan Temporary File:

- Dapat digunakan multiple kali
- Dapat di-inspect untuk debugging
- Tidak ada batasan buffer size
- Dapat diproses oleh tools yang tidak support stdin

1.8.5 stderr Handling: Kapan Redirect, Kapan Buang?

1. **Redirect stderr ke file** (`2> error.log`):

- Production scripts (audit trail)
- Debugging intermittent issues
- Compliance/regulatory requirements

2. **Buang stderr** (`2> /dev/null`):
   - Error yang expected dan tidak penting (contoh: `find` permission denied)
   - Cleaning up script output untuk user
   - Known false positives
3. **Gabung dengan stdout** (`2>&1`):
   - Ingin satu log file unified
   - Perlu preserve urutan kronologis output dan error
   - Pipeline processing (pipe perlu keduanya)

Tips

Dalam production scripts, selalu catat stderr ke file terpisah minimal saat testing. Setelah yakin stabil, baru pertimbangkan untuk membuang stderr yang tidak relevan.

1.9 Best Practices I/O Redirection

1. **Selalu cek hasil redirection**: Gunakan `$?` atau `PIPESTATUS` untuk memverifikasi sukses
2. **Gunakan noclobber untuk safety**: Aktifkan `set -o noclobber` dalam script penting
3. **Log dengan timestamp**: Gunakan `$(date)` untuk mencatat waktu dalam log
4. **Pisahkan stdout dan stderr**: Untuk debugging yang lebih mudah
5. **Gunakan tee untuk visibility**: Saat perlu melihat dan menyimpan output
6. **Cleanup file descriptor**: Tutup FD custom setelah selesai
7. **Quote variables**: Gunakan quotes untuk menghindari word splitting

```
#!/bin/bash
set -euo pipefail  # Exit on error, undefined var, pipe
   failure
```

```
LOGFILE="process-$(date +%Y%m%d).log"

# Fungsi logging
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "
   $LOGFILE"
}

# Eksekusi dengan logging
log "Proses dimulai"

if process_data > output.txt 2>> error.log; then
    log "Proses berhasil"
else
    log "ERROR: Proses gagal dengan kode $?"
    exit 1
fi

log "Proses selesai"
```

Kode 1.32: Best practice example

1.10 Rangkuman

- **File Descriptor**: Integer yang merepresentasikan I/O stream (0=stdin, 1=stdout, 2=stderr)
- **I/O Redirection**: Mengalihkan input/output menggunakan operator <, >, », 2>, dll.
- **Pipe**: Menghubungkan stdout satu perintah ke stdin perintah lain dengan operator |
- **tee**: Menampilkan output di terminal sekaligus menyimpan ke file
- **/dev/null**: Null device untuk membuang output yang tidak diperlukan
- **Kombinasi Operator**: Dapat dikombinasikan untuk workflow yang kompleks

1.11 Latihan

Latihan 3.1

Buatlah script yang:

1. Menampilkan daftar 10 file terbesar di direktori `/var/log/`
2. Menyimpan hasilnya ke file `large-logs.txt`
3. Menampilkan output juga di terminal menggunakan `tee`
4. Menangani error dengan redirect ke `error.log`

Latihan 3.2

Buat pipeline yang:

1. Membaca `/etc/passwd`
2. Mengekstrak username (kolom pertama)
3. Mengurutkan alfabetis
4. Menyimpan ke file `sorted-users.txt`

Hint: Gunakan `cut`, `sort`, dan operator redirect.

Latihan 3.3

Tulis script monitoring yang:

1. Mencatat penggunaan CPU dan memory setiap 5 detik
2. Menyimpan log dengan timestamp
3. Berjalan selama 1 menit (12 iterasi)
4. Output ditampilkan di terminal DAN disimpan ke file

Latihan 3.4

Buat perintah yang:

1. Mencari semua file `.conf` di sistem
2. Membuang pesan "Permission denied"
3. Menghitung jumlah file yang ditemukan
4. Menyimpan daftar path lengkap ke file

Latihan 3.5

Implementasikan script backup yang:

1. Menggunakan `tar` untuk backup direktori
2. Menampilkan progress dengan `tee`
3. Mencatat stdout ke `backup-success.log`
4. Mencatat stderr ke `backup-error.log`
5. Menambahkan timestamp di setiap log entry

1.12 Referensi Tambahan

- Man pages: `man bash` (section REDIRECTION)
- Man pages: `man tee`, `man mkfifo`
- Advanced Bash-Scripting Guide: I/O Redirection
- Linux Documentation Project: Bash Guide for Beginners